

Introducción a GitHub y repositorios remotos

3. Introducción a GitHub y Repositorios Remotos

¿Qué es un repositorio remoto?

Una copia de tu repositorio de Git alojada en un servidor de internet: réplica completa de commits, ramas y etiquetas. Usaremos GitHub como servidor principal del curso.

Colaboración

Varios desarrolladores trabajan sobre el mismo proyecto y comparten cambios.

Copia de seguridad

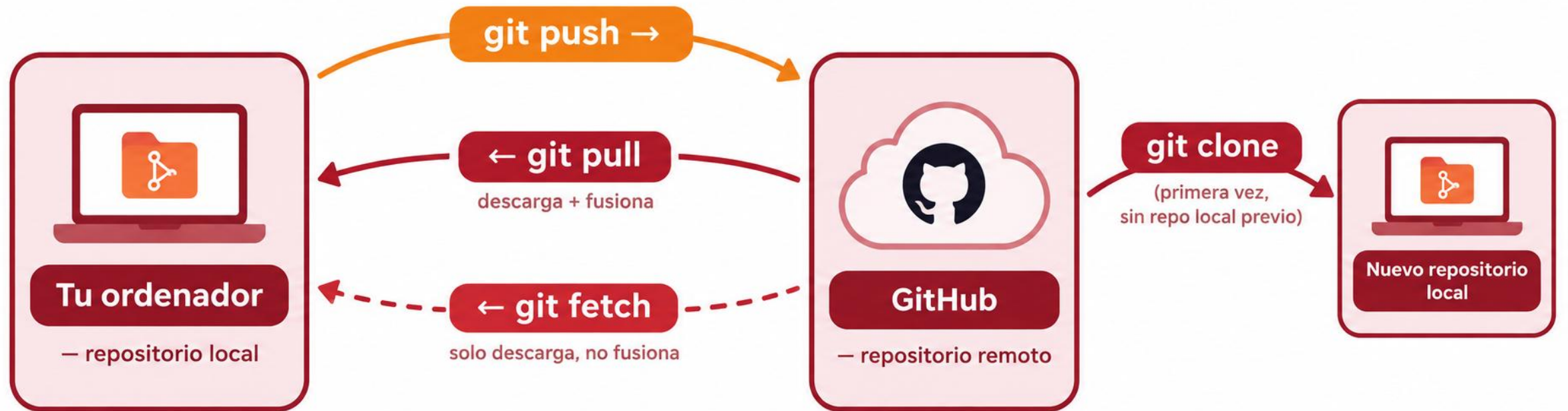
Si tu ordenador falla, el código sigue en la nube.

Distribución

Portafolio, o conectarlo a servidores de despliegue.

Alternativas a GitHub: GitLab (open source, DevOps) y Bitbucket (Atlassian, Jira/Trello).

¿Qué es un repositorio remoto?



Registro en GitHub

El proceso es gratuito

1. Sign up

github.com, arriba a la derecha.

2. Datos

Email, contraseña y usuario.

3. Verificación

Puzzle + código al correo.

4. Dashboard

Ya puedes crear y explorar repositorios.

Desde el dashboard verás el listado de repositorios (vacío al principio) y el botón verde «New» para crear uno.

Autenticación y conexión en GitHub

Antiguamente se usaba la contraseña de la cuenta, pero ya no está permitido desde la terminal. Hoy hay tres formas de verificar tu identidad.

Token de acceso personal (PAT)

Una «contraseña larga» para repositorios, revocable. El método más habitual al empezar.

Claves SSH

Par de claves criptográficas en tu ordenador. Más profesional y cómodo a largo plazo.

GitHub CLI (gh)

Herramienta oficial que autentica a través del navegador.

El PAT es más rápido al principio; SSH es más seguro y no requiere pegar ningún token cada vez que haces push. En profesional, SSH es el estándar.

Generando un Token de Acceso Personal (PAT)

1. Settings

Tu foto de perfil → Settings.

2. Developer settings

Menú lateral, abajo del todo.

3. Tokens (classic)

Personal access tokens → Tokens (classic).

4. Generate new token

Marca el permiso repo, y Generate token.

Cópialo ahora: es la única vez que se muestra

Si cierras la pestaña sin copiarlo, tendrás que generar uno nuevo. Empieza por ghp_... y se pega como contraseña al hacer git push.

Configurando la autenticación por SSH

1-2. Generar la clave y copiar la pública

```
ssh-keygen -t ed25519 -C "tu_correo@ejemplo.com"

# Windows (Git Bash)
clip < ~/.ssh/id_ed25519.pub

# Mac
pbcopy < ~/.ssh/id_ed25519.pub
```

Dos archivos: privada y pública

La privada nunca se comparte. La pública (empieza por ssh-ed25519) es la que le das a GitHub.

3. Añadir la clave a GitHub

Settings → SSH and GPG keys

New SSH key

Título + pegar la clave

Ej. «Portátil clase»

Add SSH key

Push y pull sin pedir credenciales

A partir de ahora, clona con la URL SSH (git@github.com:...) en vez de HTTPS.

Subiendo tu proyecto: crear repo y enlazar

1. Creando el repositorio en GitHub

Botón New, dale un nombre. Si ya tienes un repo local listo, no marques README ni .gitignore: necesitas un repositorio vacío.

2. Enlazando el local con el remoto: `git remote add`

```
# HTTPS
git remote add origin https://github.com/tu-usuario/tu-repo.git

# SSH
git remote add origin git@github.com:tu-usuario/tu-repo.git

git remote -v    # comprobar que se ha creado bien
```

origin es un apodo para la URL del servidor remoto — por convención se llama siempre así, así no escribes la URL completa cada vez.

El primer push

-u (--set-upstream) vincula tu rama local con la remota

```
git push -u origin main
```

A partir de ahí

Git sabe adónde ir cuando ejecutas git push o git pull a secas, sin escribir origin main cada vez.

Nunca uses git push --force salvo que sepas lo que haces

Sobreescribe el historial del servidor y puede borrar el trabajo de tus compañeros. Si hay conflicto, primero git pull para integrar, y luego push. --force solo es aceptable en una rama propia que nadie más usa.

Si pide usuario y contraseña, pega el Token PAT como contraseña. Si no pide nada, tienes SSH o el Credential Manager de Windows ya recordaba tu sesión.

Clonar un repositorio desde cero: git clone

Llegas a un ordenador sin nada del proyecto: en vez de git init

```
git clone https://github.com/tu-usuario/tu-repo.git
# O con SSH:
git clone git@github.com:tu-usuario/tu-repo.git
```

1. Crea la carpeta

Con el nombre del repositorio.

2. Descarga todo

Commits, ramas y archivos.

3. Configura origin

Apuntando al remoto,
automáticamente.

La URL de clonación se obtiene desde el botón verde «<> Code» del repositorio en GitHub.

Comprobar y descargar: fetch frente a pull

git fetch — comprobar sin descargar de verdad

```
git fetch
```

git pull — fetch + merge automático

```
git pull
```

	git fetch	git pull
Conecta al remoto	✓	✓
Descarga cambios	✓	✓
Modifica tus archivos	✗	✓
Cuándo usarlo	Revisar antes de integrar	Actualizar directamente

¿Y si git pull genera un conflicto?

Si tú y alguien más (o tú desde la web) habéis modificado el mismo archivo, git pull no puede fusionar solo. Se resuelve igual que en ramas: edita las marcas, git add, git commit.

Las ramas en el repositorio remoto

Publicar una rama en GitHub

```
git switch -c nueva-funcionalidad  # crea la rama y cambia a ella
# ... trabajas, haces commits ...
git push -u origin nueva-funcionalidad  # la primera vez
git push                                # las siguientes veces
```

Solo existen en tu ordenador hasta que las subes

Todo lo que aprendiste sobre ramas funciona igual en el remoto — el selector de ramas de GitHub las mostrará todas las subidas.

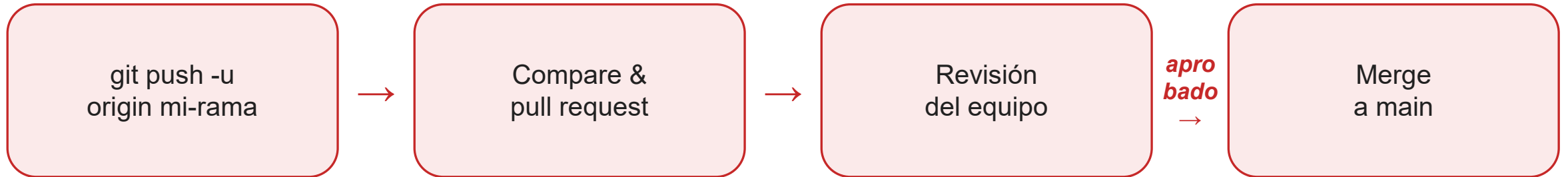
Borrar ramas en remoto (una vez fusionada y borrada en local)

```
git push origin --delete nueva-funcionalidad
```

También puedes hacerlo desde la web: Branches → icono de papelera.

Pull Requests: proponer cambios para revisión

Una PR es una petición en GitHub para que alguien revise tu rama antes de fusionarla en main. En lugar de fusionar directamente, propones los cambios y el equipo los discute y aprueba.



Files changed

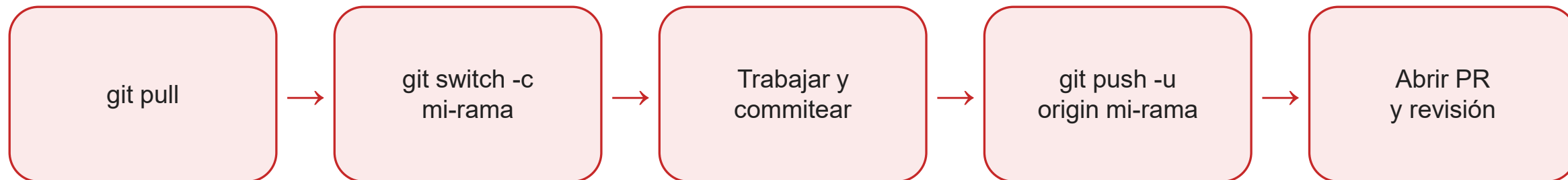
Líneas añadidas en verde, borradas en rojo.

No es solo para equipos

Da historial de decisiones también en proyectos propios.

Si no se aprueba, se piden cambios y se hace push de nuevo a la misma rama: la PR se actualiza sola.

Flujo de trabajo habitual



1. Actualizarte

```
git pull
```

2. Crear una rama

```
git switch -c mi-rama
```

3. Trabajar y commitear

```
git add . / git commit -m "..."
```

4. Publicar la rama

```
git push -u origin mi-rama
```

5. Abrir una Pull Request

y esperar revisión antes de fusionar

Integración de Git y GitHub en IntelliJ IDEA

Soporte nativo para Git y GitHub

La mayoría de operaciones se pueden hacer desde la interfaz, sin usar la terminal.

Commit

Panel para marcar archivos al staging, mensaje y diff visual.

Push (Ctrl+Shift+K)

Muestra los commits pendientes y hacia qué rama van.

Pull / Fetch

Descarga e integra cambios del remoto.

Branches

Ramas locales y remotas, New Branch, cambiar con un clic.

La terminal y el IDE son complementarios: la terminal da control total; el IDE acelera las operaciones del día a día.

Ideas clave

1

`git remote add origin <url>` vincula el repositorio local con el remoto; origin es el alias de esa URL.

2

`git push -u origin <rama>` sube la rama por primera vez y la vincula; después, `git push` a secas.

3

`git clone <url>` descarga un repositorio completo en una carpeta nueva.

4

`git fetch` descarga sin tocar tus archivos; `git pull` descarga y fusiona.

5

`git push origin --delete <rama>` borra una rama del remoto.

6

Una Pull Request es una propuesta de fusión que el equipo revisa antes de integrar en main.

7

El flujo habitual: pull → nueva rama → trabajar y commitear → push → PR → revisión y merge.

8

PAT y SSH son las dos formas principales de autenticarte; SSH es el estándar profesional.